



REPTE PARADIGMES DE LA PROGRAMACIÓ I BASES DE DADES

Seminari d'Intel·ligència artificial

Al. Eric Torrontera Ruiz

Splatoon

El joc consta d'un tauler $N \times M$ posicions. L'objectiu del joc és aconseguir pintar el màxim de caselles, donat que guanya el jugador que pinti més que l'oponent.

Regles:

1. Els jugador humà pinta les caselles per les que passa de color vermell i el jugador IA de color groc.
2. Els jugadors no poden passar per caselles ja pintades, o "cremades" (color gris) **Les cremades ara són caixes.**
3. La posició inicial ha de situar (de forma aleatòria) a cada jugador en caselles no coincidents.
4. El jugador humà i la IA no poden ocupar la mateixa casella alhora.
5. Un jugador no pot saltar el seu torn de moure.
6. La màquina ha de sortejar qui comença la partida.
7. El taulell s'inicia amb un nombre d'obstacles (caselles cremades) situats aleatòriament.
8. Els jugadors es mouen en vertical o en horitzontal, i amb una distància d'una sola posició. No estan permesos els moviments en diagonal.
9. El joc finalitza quan no queda cap casella per pintar, quan un jugador ha pintat prou caselles per no ser superat pel contrincant, o quan mor per ofegament (no pot moure a cap casella lliure).
10. En cas d'exhaurir les caselles lliure, i que ambdós jugadors obtinguin el mateix número de pintades, la partida acabarà en taules.

En aquest treball hem utilitzat el següent codi font de GitHub:

https://github.com/pepoencamp/Repte_M4_2022_UDA ⁽¹⁾

(1) Autor del codi font inicial: Josep Ribó Ferriz: Associate Professor - Universitat d'Andorra

Jocs amb adversari: Minimax

Quant l'algorisme Minimax() hem realitzat diverses modificacions i millores respecte al codi original. Ara utilitza el nombre de jugades (profunditat de cerca), com a modificador que premia/castiga les jugades llargues (i el risc), en/a contra/favor de les jugades curtes. IA sempre és Max, per tant hem definit una heurística negativa per PLAYER, i una heurística positiva per IA.

Utilitzarem la següent funció com a **funció d'utilitat**: $+2 + \text{"DEPTH"}$ si és un estat on guanya IA, $-2 - \text{"DEPTH"}$ si és un estat on ha guanya PLAYER, 0 en cas d'empat. Per posar un exemple si la partida comença amb DEPTH_LIMIT = 10 i troba una solució guanyadora a DEPTH = 5 la funció d'utilitat retornarà 7.

"Bugs" Codi Inicial

BUG 01: Hem trobat que en alguns cassos del joc IA es desorientava. Era fet que quan IA guanyava una partida ofegant a Player sense moviments, IA no retornava cap valor de "row" i "col". Hem corregit aquesta casuística amb valors "None".

```
if turn == AI and not game_over: # Espera la jugada d'IA
    row, col, minimax_score = minimax(board, DEPTH_LEVEL, float('-inf'), float('inf'), True, heatmap) # AI tanto amb
    pygame.time.wait(500)

    if row is not None: # Correcció d'error on IA es desorienta perquè no rep cap valor i guanya sense mov
        moure_fitxa(board, ROW_COUNT-row-1, col, AI_PIECE)

    if jugada_guanyadora(board, AI_PIECE):

        if row is not None: # Correcció moviment fitxa WINNER a una pos inexistente
            moure_fitxa(board, ROW_COUNT-row-1, col, WINNER_PIECE)
        else:
            for c in range(COLUMN_COUNT): # Compta la puntuació de les fitxes de cada jugador, de forma separada
                for r in range(ROW_COUNT):
                    if (board[r][c] == AI_PIECE): # Hem de buscar on es la fitxa guanyadora
                        board[r][c] = WINNER_PIECE

            mostra_tauler(board)
            dibuixa_tauler(board)

            label = myfont.render("Guanya IA!", 1, BLUE)
            screen.blit(label, (40,10))
            pygame.display.flip()
            game_over = True
    else:
        mostra_tauler(board)
        dibuixa_tauler(board)
        turn += 1
        turn = turn % 2

if game_over:
    pygame.time.wait(3000)
```

Jocs amb adversari: Decisions imperfectes

L'algorisme MINIMAX té un cost exponencial, ja que ha de recórrer tots els nodes de tots els nivells abans de prendre una decisió. Si la cerca es fa a una profunditat d i tenim un factor de ramificació de b , minimax té un cost $O(b^d)$. Hem programat la poda α - β que reduirà en nombre el nombre de nodes a considerar, en el cas ideal $O(b^{d/2})$.

Tot i que amb el mateix temps ara podem explorar el doble de profunditat, a partir de certs nivells, el temps de cerca dels nodes terminals encara és presenta mol elevat. Per aquest motiu caldrà limitar la profunditat de l'arbre de cerca. Aquesta modificació forçarà la funció Minimax() a calcular una funció heurística avaluat_estat() de les posicions que estan al límit de profunditat establert, que no són terminals, al mateix temps que tindrà en compte la funció d'utilitat pels nodes terminals amb una millor qualificació si IA guanya.

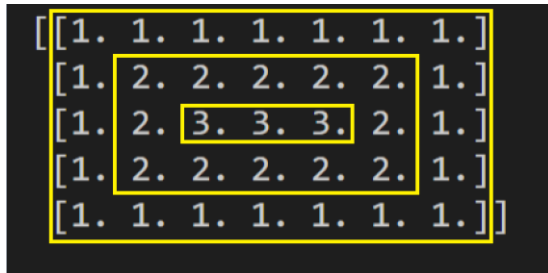
Per exemple, parar l'expansió $N = 4$ sense haver arribat al node terminal, la nostra heurística calcularà un indicador de "qualitat" del tauler. En aquest cas hem fet us d'un "heatmap" com a matriu d'avaluació heurística dinàmica, on les caselles interiors tenen millor valoració que les exteriors

"Heatmap"

El que farà el nostre algoritme serà calcular el nombre de punts de cada jugador en relació al "heatmap". Si IA té un nombre de punts superior a PLAYER retornarà (+1). Altrament si PLAYER té un nombre de punts superior a IA retornarà (-1). Finalment si IA i PLAYER tenen el mateix nombre de punts, retornarà un valor en funció del risc. Retornarà (+1) si el següent en jugar és IA i (-1) si el següent en jugar és PLAYER.

```
--Heatmap--
[[1. 1. 1. 1. 1. 1. 1. 1. 1. 1. 1. 1.]
 [1. 2. 2. 2. 2. 2. 2. 2. 2. 2. 2. 1.]
 [1. 2. 3. 3. 3. 3. 3. 3. 3. 3. 2. 1.]
 [1. 2. 3. 4. 4. 4. 4. 4. 4. 3. 2. 1.]
 [1. 2. 3. 4. 4. 4. 4. 4. 4. 3. 2. 1.]
 [1. 2. 3. 3. 3. 3. 3. 3. 3. 3. 2. 1.]
 [1. 2. 2. 2. 2. 2. 2. 2. 2. 2. 2. 1.]
 [1. 1. 1. 1. 1. 1. 1. 1. 1. 1. 1. 1.]]

-----
[[0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0.]
 [0. 6. 6. 0. 0. 0. 0. 0. 0. 0. 0. 0.]
 [0. 0. 0. 6. 0. 0. 0. 0. 6. 0. 0. 0.]
 [0. 0. 0. 0. 0. 0. 0. 6. 0. 0. 0. 0.]
 [0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0.]
 [0. 0. 0. 2. 0. 0. 0. 6. 6. 0. 0. 0.]
 [0. 0. 0. 0. 0. 6. 0. 0. 0. 0. 0. 0.]
 [1. 0. 0. 6. 0. 0. 0. 0. 0. 0. 0. 0.]]
```



Per tal de crear la matriu headmap hem creat un algorisme recursiu que de forma elegant i eficient realitza un recorregut per tota la matriu en espiral (La matriu pot ser un rectangle) que insereix valors més elevats en funció de la profunditat.

El pseudocodi que hem emparat per recrear un headmap és el següent:

PROFUNDITAT $\leftarrow 3$

MENTRE PROFUNDITAT > 0 FER

Moviment a la dreta.

Límit inferior files + 1

Moviment cap a baix

Límit columnes superior - 1

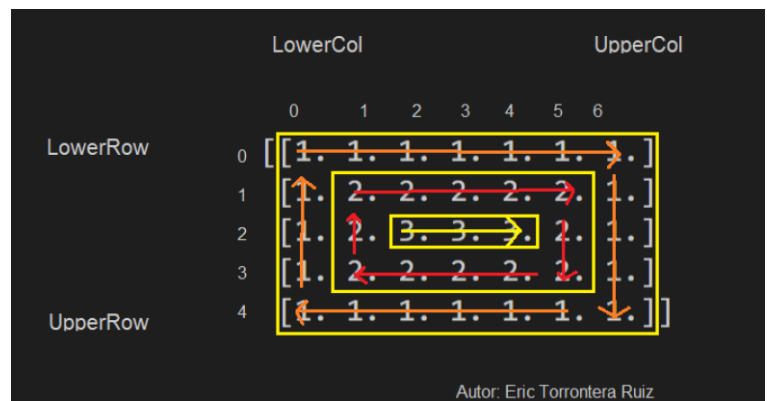
Moviment a la esquerra

Límit superior files -1

Moviment cap a dalt

Límit columnes superior -1

FI_MENTRE

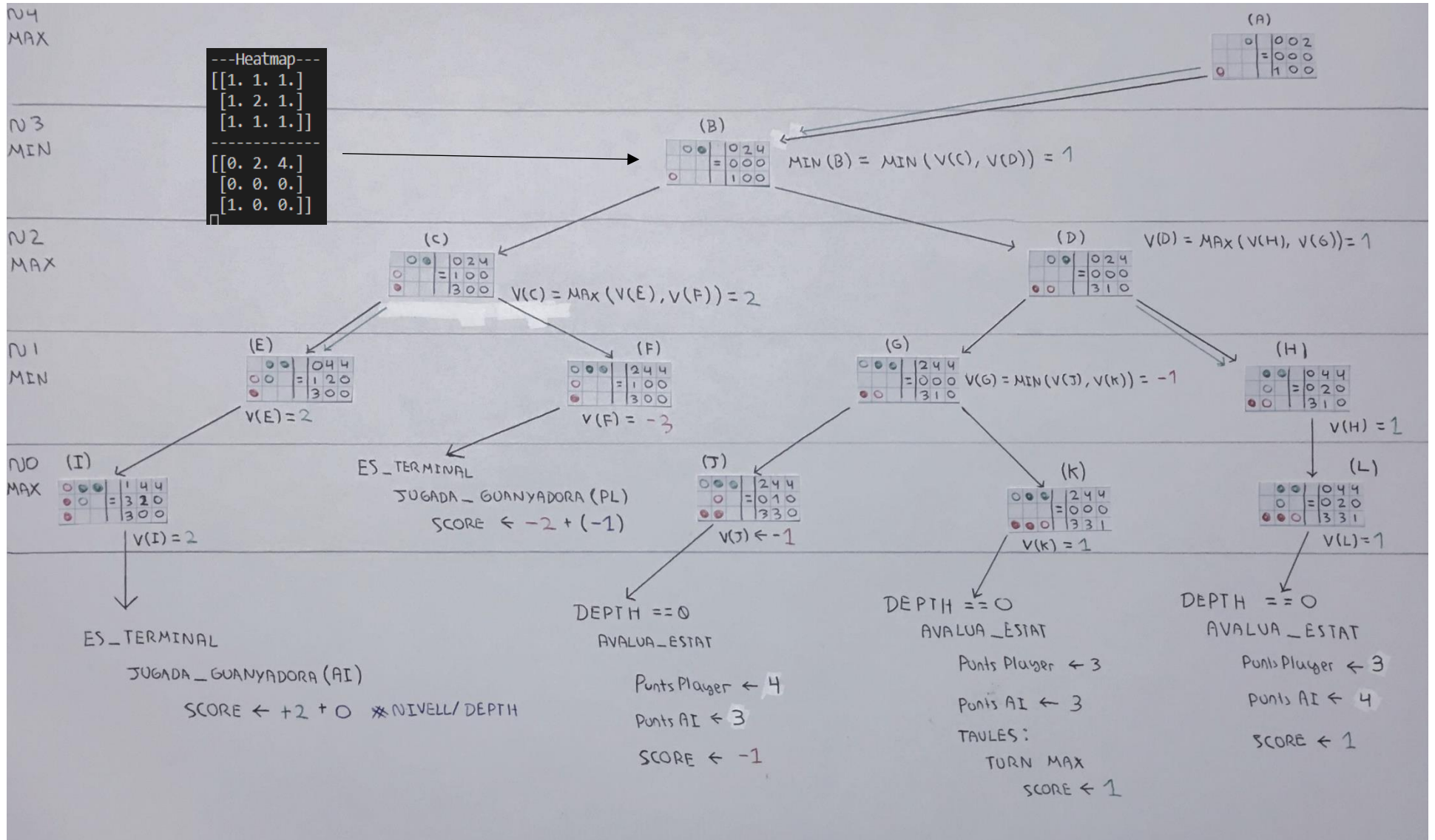


A continuació s'exposa un exemple d'una de les branques que agafaria l'algoritme minimax en un taulell 3x3 on comença IA. pareixen totes els cassos terminals i no terminals del l'inici d'una partida d'Splatoon. Comença IA (Una branca) i amb una profunditat de cerca amb $N = 4$

Al estat **K** la nostra heurística ha determinat una probabilitat en favor d'IA ja que tot i que hi ha taules, el següent en moure és IA

Al estat **J** la heurística proposada resol el conflicte a favor de PLAYER ja que la seva puntuació del heatmap és superior (Té la casella central)

Altrament comentar que el fet de fitar la cerca, provoca que les decisions que es prenen no són òptimes ja que només reflecteix la probabilitat de guanyar.



NIVELLS DE DIFICULTAT

Per afegir l'opció de jugar amb diferents nivells de dificultat contra una intel·ligència artificial en partides de dos jugadors tenim diverses formes per procedir.

1. Limitar la profunditat de cerca quant als nivells. Per posar un exemple, si la IA només té en compte 2 o 3 nivells de dificultat, té més probabilitats d'agafar un camí on PLAYER té l'opció de fer un moviment cap a una branca guanyadora.
2. Limitar la profunditat de cerca establint un màxim temps si fem us de l'algorisme tot temps.
3. Canviar la funció d'utilitat per que IA vagi sempre cap a taules o perquè els nodes guanyadors no tinguin en compte la profunditat on es troben per donar espai al jugador a realitzar alguna jugada guanyadora.
4. Afegir jugades imperfectes amb element d'atzar per que IA tingui una probabilitat X d'agafar un camí que no és òptim.
5. Utilitzar altres heurístiques amb menys o més eficàcia per avaluar nodes no terminals.

Altrament comentar que com que els jugadors no poden moure's a caselles pintades o obstacles i l'ordre de càlcul de l'arbre es veu reduït en fer jugades. Podríem jugar amb això per tal de incrementar la dificultat. En el nostre cas he programat pels taulers de més de 3 x 3 amb dificultat Fàcil o Normal que aparegui 1 posició cremada per cada 10 caselles i una de més de per la dificultat Difícil.

En aquest treball he optat per fer 5 nivells de dificultat:

Fàcil (1): Intermitentment IA fa una jugada aleatòria seguida d'una jugada amb la funció d'utilitat descrita abans + heurística a partir del nivell 5. La primera jugada d'IA sempre és aleatòria

IA_Aleatori – PLAYER – IA_Normal – PLAYER – IA_Aleatori ...

Normal (2): Jugades IA amb la funció d'utilitat descrita abans + heurística a partir del nivell de la profunditat 5.

Difícil (3): Jugades IA amb la funció d'utilitat descrita abans + heurística a partir del nivell de la profunditat 9.

Programable (4): Límit de profunditat de cerca, component aleatori i posicions cremades definides per les constants.

Modo Dios (5) : Jugades IA amb la funció d'utilitat descrita abans + heurística a partir del nivell de la profunditat 100. Per un tauler 6x8 les jugades inicials triguen minuts

```
#####  
#                                                                 #  
#                               DEFINICIO DE MODULS  (IA)          #  
#                                                                 #  
#####  
  
def Nivells(nivell):  
    if nivell == 1:  
        depth = 5  
        guess = True  
        burned = math.floor((Constants.COLUMN_COUNT*Constants.ROW_COUNT) / 10) - 1  
    elif nivell == 2:  
        depth = 6  
        guess = False  
        burned = math.floor((Constants.COLUMN_COUNT*Constants.ROW_COUNT) / 10) - 1  
    elif nivell == 3:  
        depth = 9  
        guess = False # Utilitzem floor per tal de poder jugar bé sense cremades al tauler 3x3  
        burned = math.floor((Constants.COLUMN_COUNT*Constants.ROW_COUNT) / 10)  
    elif nivell == 4:  
        depth = Constants.DEPH_LEVEL  
        guess = False  
        burned = Constants.BURNED_POSITIONS  
    elif nivell == 5:  
        depth = 100  
        guess = False  
        burned = 9
```

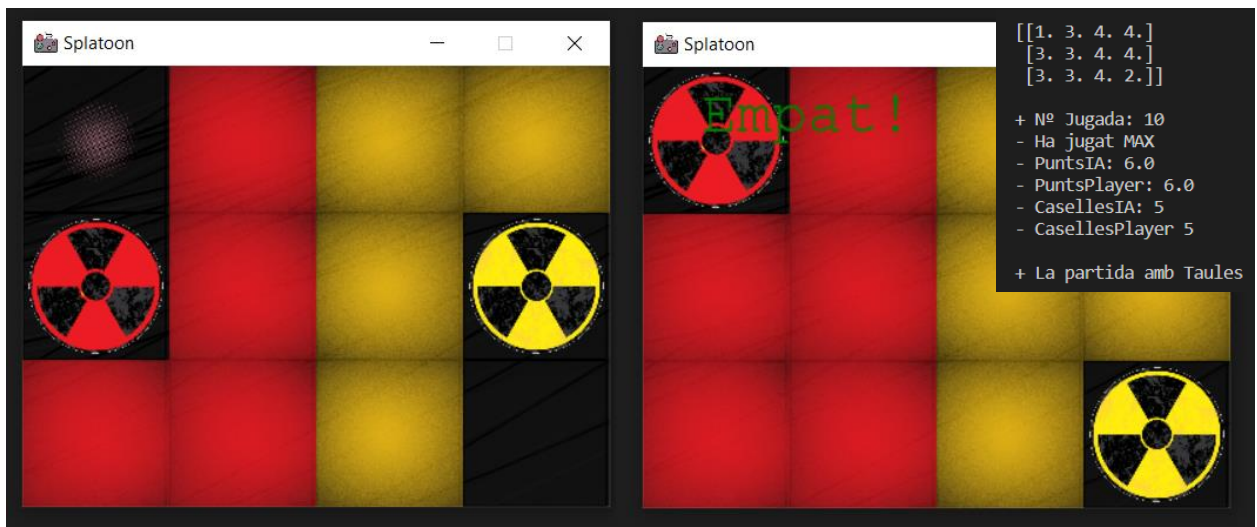
PROVES UNITARIES

REGLA 10: "En cas d'exhaurir les caselles lliure, i que ambdós jugadors obtinguin el mateix número de pintades, la partida acabarà en taules."

Empat I – On IA Comença primer (PARELL)



Empat II – On PLAYER Comença primer (PARELL)



TAULA DE PUNTUACIONS PER NIVELL

Mode	Guanya Player	Guanya IA
Fàcil	Punts Caselles Heatmap PL * 1	-(Punts Caselles Heatmap IA * 1)
Normal	Punts Caselles Heatmap PL * 2	-(Punts Caselles Heatmap IA * 2)
Difícil	Punts Caselles Heatmap PL * 3	-(Punts Caselles Heatmap IA * 3)
Programmer	Punts Caselles Heatmap PL * 4	-(Punts Caselles Heatmap IA * 4)
Modo Dios	Punts Caselles Heatmap PL * 5	-(Punts Caselles Heatmap IA * 5)

Si una partida queda en taules la puntuació per ambdós és 0. Altrament comentar que he afegit les “puntuacions negatives” per jugades no guanyadores de Player per visualitzar través de MySQL partides llargues, per nivell, no guanyadores.

*QUERY: SELECT * FROM Partides WHERE PUNTUACIO < 0 ORDER BY PUNTUACIO*

	ID_PARTIDA	NICK	ID_NIVELL	PUNTUACIO
▶	14	Desconegut	4	-96
	8	Desconegut	4	-84
	12	Desconegut	4	-72
	10	Desconegut	4	-60
	2	Whiterose	4	-8
	13	Desconegut	4	-8
•	NULL	NULL	NULL	NULL